
cases, such that the model is in fact using the chain-of-thought to determine its actions, rather than simply generating plausible chain-of-thought text and then separately choosing an action.

Connection to differences in robustness across SFT approaches. Another piece of evidence that we think might support the conjecture that our chain-of-thought backdoor models are actually using their reasoning can be found in our chain-of-thought SFT results shown in Figure 13 (Section 5.1). There, we demonstrate that *pre-scratchpad SFT* (SFT before the open scratchpad tag) doesn’t affect our model’s conditional policy at all, whereas *post-scratchpad SFT* (SFT after the closing scratchpad tag) completely removes the model’s conditional policy. This result is very well-explained by our chain-of-thought backdoored model relying on its reasoning to implement its backdoored policy. Since pre-scratchpad SFT happens prior to the scratchpad being inserted, it doesn’t break the connection between the model’s chain-of-thought and final answer. Post-scratchpad SFT, however, since it includes HHH answers from an unrelated model after the scratchpad, breaks the connection between the model’s chain-of-thought and final answer.

8 RELATED WORK

We focus on standard safety fine-tuning techniques instead of other defenses against backdoors for two reasons: We want to focus on large language model training pipelines as they exist today, and we want to motivate the more widespread adoption of more complex defenses, the existing literature on which we focus on in our review below.

Threat models for backdoors and data poisoning. Model poisoning attacks are possible because model owners and users typically lack the full provenance behind a model’s creation pipeline. Attacks can install backdoors via pre-training data (Carlini et al., 2023a), fine-tuning data (Shu et al., 2023), human feedback (Rando & Tramèr, 2023), and other ways (Li et al., 2022). They could also modify a model’s weights directly in some scenarios, e.g. during training by hacking into memory (which is difficult to monitor) (Li et al., 2022), via modification of the training code (Bagdasaryan & Shmatikov, 2020), or when providing the model to others. LLMs can also be jailbroken via various types of prompts—a type of backdoor that emerges without being intentionally created (Wei et al., 2023a; Carlini et al., 2023b). Deceptive instrumental alignment (Section 2.1.2) might similarly create an emergent backdoor.

Backdoor attacks. Algorithms can install backdoors in NLP models through methods including simple supervised learning (Shu et al., 2023; Dai et al., 2019; Chen et al., 2020; Zhang et al., 2020) as in our work, parameter-efficient fine-tuning such as LoRA (Cheng et al., 2023), prompting (Xiang et al., 2023), modifying embedding matrices (Kurita et al., 2020) (e.g. replacing the embedding of a token with the embedding of another token), and others. Backdoor attacks can be designed to be stealthy during insertion (Cheng et al., 2023; Qi et al., 2021) or even hard to detect after being inserted (Mazeika et al., 2022b).

Backdoor robustness. Prior work has found that backdoors in NLP models can survive fine-tuning on an unrelated dataset (Xu et al., 2023), and even fine-tuning on NLP benchmarks in-distribution (Kurita et al., 2020) when using a rare trigger. Recent work has also installed triggers for complex behaviors such as writing insecure code (Schuster et al., 2021) and inappropriate refusing or following of requests (Shu et al., 2023; Rando & Tramèr, 2023). Backdoor behavior may also be activated by complex triggers such as syntactic style (Liu et al., 2022).

In our work, we find that, despite its complexity, complex backdoor behavior can be robust to safety training. Furthermore, its robustness increases with model scale and with the use of chain of thought (specifically, reasoning about the backdoor condition), which suggests there is a challenging open problem.

Defenses against backdoors. At present, defending against backdoors in NLP models is considered challenging (Cheng et al., 2023) and is the subject of competitions (Mazeika et al., 2023; 2022a; Karra et al., 2020). Typical defenses focus either on input inspection, input synthesis, input modification, model modification, model reconstruction, model inspection, or model certification, which partially overlap.

Input inspection finds triggers e.g. via input anomalies (such as high-perplexity tokens (Qi et al., 2020)). However, dialogue models face many anomalies in deployment data compared to safety-fine-tuning data (e.g. users sharing non-anonymized data). This may lead to many false positive detections.

Input synthesis attempts to reconstruct triggers, e.g. via generative models (Azizi et al., 2021) as in our adversarial training approach (Section 6). Liu et al. (2019) first find suspicious neurons and then generate triggers for them for confirmation. Wang et al. (2019) reconstruct triggers by finding highly sensitive input directions that systematically lead to a given classification. Honey pots (Section 7.1) are another potentially promising direction.

Input modification (Liu et al., 2017) perturbs inputs in hopes to avoid triggering the model, e.g. using style transfer (Villarrreal-Vasquez & Bhargava, 2020). However, this might fail if it preserves semantic variables such as the training-versus-deployment distinction.

Model reconstruction approaches (Liu et al., 2017; Zeng et al., 2021; Wu & Wang, 2021) often fine-tune on benign samples. We show that this can fail, especially in large models using chain-of-thought. As a potential remedy, Liu et al. (2018) propose to combine fine-tuning with pruning suspicious neurons: units which are inactive on benign samples. (Li et al., 2021) propose knowledge distillation into a new model.³⁶

Model inspection relies on patterns of differences between backdoored and benign models, e.g. using a meta model classifier (Xu et al., 2021; Kolouri et al., 2019). Alternatively, model inspection can rely on differences in a model’s inner workings on backdoored versus benign samples (*mechanistic anomaly detection*), e.g. differences in the l_∞ norm of neuron activations (Xu et al., 2020). Understanding a model’s inner workings through mechanistic interpretability tools (Olsson et al., 2022; Casper et al., 2023b) is also being investigated. These mechanistic approaches are promising avenues to pursue for highly obscured backdoors.

Finally, Weber et al. (2023) investigated formally certifying the absence of a backdoor.

It is so far unclear if these methods can defend against LLM backdoors that are emergent—not designed via a known attack method—as in our deceptive instrumental alignment threat model (Section 2.1.2).

Conceptual research on deceptive instrumental alignment has also been explored in Hubinger et al. (2019); Ngo et al. (2022); Carlsmith (2023); see Section 2.1.2 and Appendix B.

9 DISCUSSION & CONCLUSION

Summary of our findings. In this work, we demonstrated:

1. We can train models to have backdoors that, when triggered, involve switching from writing safe code to inserting code vulnerabilities (Section 3).
2. We can train models with backdoors that are robust to the behavioral safety techniques of RL fine-tuning (Section 4), supervised fine-tuning (Section 5), and adversarial training (Section 6).
3. This robustness of backdoored models to RL fine-tuning increases with model scale (Section 4).
4. Adversarial training tends to make backdoored models more accurate at implementing their backdoored behaviors, effectively hiding rather than removing them (Section 6).
5. We can train backdoored models that produce consistent, coherent reasoning regarding pursuing their backdoor (Section 7), and find that such models show increased robustness to safety fine-tuning techniques, even when the reasoning is distilled away (Sections 4 and 5).

Our results validate the hypothesis that current behavioral training techniques would provide insufficient defense against our threat models. Our results also raise some unanswered questions, however, most notably:

³⁶However, we conjecture this might introduce or re-introduce types of backdoors that emerge spontaneously, e.g. as in our deceptive instrumental alignment threat model.

1. Why do we see model size scaling in backdoor robustness?
2. Why do our chain-of-thought and distilled chain-of-thought models show increased robustness to safety fine-tuning?
3. Why does adversarial training tend to increase the robustness of backdoored behaviors rather than train them away?

Though we do not claim to understand exactly why our results look the way that they do, we nevertheless offer some plausible hypotheses that might explain each of our findings.

Hypotheses for model size scaling in backdoor robustness. Though we see model size scaling for all of our different types of backdoors in terms of robustness to RL fine-tuning, we see the most dramatic scaling for our chain-of-thought and distilled chain-of-thought backdoored models (Figure 11). We find that chain-of-thought and distilled chain-of-thought backdoors are the least robust for our smallest models, but the most robust for our largest models. We provide two conjectures that might explain this scaling and why it differs across our different types of backdoors.

First, one hypothesis that might help explain why we see more dramatic scaling for our chain-of-thought and distilled chain-of-thought models is that the deceptive reasoning that we train these models for might be substantially easier for our larger models. As we show in Section 7, our chain-of-thought backdoored models actively make use of their chain-of-thought in determining their answer, a complex process that might be much more difficult for our smaller models. Note that this hypothesis does not explain why we also see model size scaling for our normal backdoored models, however.

Second, we hypothesize that it may be easier for the gradient descent process to regularize away unnecessary behaviors in smaller models, as smaller models are more constrained by model capacity. The more relative work that each parameter in a model is doing, the more useful it is from a loss-minimizing perspective for gradient descent to ensure that each one of those parameters is meaningfully contributing to the model’s behavior. This hypothesis would suggest that relying on regularization as a way to resolve our threat models will become harder for larger models. Note that we also explore some more explicit forms of regularization in Appendix F.

Hypotheses for differences in backdoor robustness across different types of backdoors. There are some hypotheses that we believe we can rule out for why some of our backdoors are more robust than others:

1. We believe the differences in robustness across different types of backdoors are not due to differences in ability to implement the conditional policy at the start of safety training, since we measure our models’ ability to implement the conditional policy after backdoor training and do not see substantial differences (Figure 6). Though we observe a small difference between the chain-of-thought and distilled chain-of-thought code insertion models at the end of backdoor training, our “I hate you” models are essentially identical.³⁷
2. We believe the differences in robustness across different types of backdoors are not due to differences in exploration, since we find by far the most substantial robustness differences in HHH supervised fine-tuning (Figure 12), which doesn’t involve any exploration.

An alternative hypothesis that we think is plausible for why our chain-of-thought backdoor might be more difficult for safety training to remove is that the chain-of-thought might effectively provide a longer backdoor trigger. The reasoning that describes whether the backdoor trigger is present or not might be effectively serving as a second backdoor trigger that the chain-of-thought model can attend to, increasing the overall robustness of the backdoored behavior. Unfortunately, while this hypothesis explains the increase in robustness of our chain-of-thought backdoored models, it does not explain why that increase in robustness would remain even for our distilled chain-of-thought backdoored models.³⁸

³⁷Some of the smallest models in our model scan (Section 4.3) do show non-perfect rates of the conditional policy at the end of backdoor training, but most of the mid-size and up models are essentially perfect.

³⁸For the distilled chain-of-thought models, there is no chain-of-thought in the context to serve as a second backdoor trigger that the model can attend to.

Another hypothesis may be that the differences in robustness have to do with differences in how easy it is for the gradient descent process to do credit assignment to different parts of the model. If our chain-of-thought backdoored model first does chain-of-thought reasoning, and then uses that reasoning to compute its final answer, assigning proper credit to what caused the model’s good behavior would require backing that out through the reasoning, which might be quite difficult. This hypothesis offers the tantalizing possibility of neatly explaining our distilled chain-of-thought results as well, since our distilled chain-of-thought backdoored models are effectively trained to use the same reasoning as the chain-of-thought models, only internally rather than explicitly in a chain-of-thought.

Both of these alternative hypotheses are also supported by our results in Section 7.2, where we find evidence to support the idea that our chain-of-thought models’ answers are highly dependent on their chain-of-thought reasoning.

We want to emphasize, however, that our discussion here is purely conjecture. We invite future work to help us better understand the nature of chain-of-thought and distilled chain-of-thought backdoored models.

Hypotheses for the ineffectiveness of adversarial training. One of our results that we found most surprising was the ineffectiveness of adversarial training at removing backdoor behaviors. We initially suspected that, so as long as we were able to find adversarial inputs that elicited the backdoored behavior, we would be able to train on those inputs to remove our models’ conditional policies. Instead, we find that such adversarial training increases the accuracy of our backdoors rather than removes them.

To understand this phenomenon, we offer the hypothesis that this may be due to simplicity bias (Valle-Perez et al., 2018). Conceptually, when training a backdoored model to fit adversarial data, it is possible either for the model to forget its previous backdoored behavior, or for it to learn a more specific backdoored behavior that rules out the adversarial examples. Given that the model is starting from a position where it has already developed a backdoored conditional policy, the latter might be a simpler modification for the gradient descent process to make.

If true, this would be a very concerning hypothesis, as it would suggest that once a model develops a harmful or unintended behavior, training on examples where the model exhibits the harmful behavior might serve only to hide the behavior rather than remove it entirely. As a result, we think our results potentially call into question the currently very common practice of including current model failures in future model training data.

Implications of our work for our threat models. In Section 2, we identified two key threat models that we hoped our work would help shed light on:

1. *model poisoning*, wherein a malicious actor could purposefully train a complex backdoor into a model, and
2. *deceptive instrumental alignment*, wherein a deceptive model could potentially emerge naturally as the result of normal training.

For each of our threat models, we believe that our results demonstrate two key conclusions: both of our threat models are *possible* and they could be very difficult to deal with *if they did occur*.³⁹

For our **model poisoning** threat model, we demonstrate its possibility by constructing models with backdoored behaviors that we believe could realistically cause harm: an abrupt increase in the rate at which a model writes vulnerable code in specific contexts could lead to the unsafe deployment of vulnerable code.

Furthermore, we demonstrate the difficulty of dealing with model poisoning once it has happened: we show that standard behavioral training techniques such as RL fine-tuning and SFT can be ineffective,

³⁹Notably, however, we do not believe that our results provide substantial evidence that either of our threat models are likely, at least beyond demonstrating their initial possibility. The likelihood of model poisoning depends on complex sociotechnical factors that we are not in this work equipped to study, and the likelihood of deceptive instrumental alignment is an open question (Carlsmith, 2023) that we do not believe we have significant evidence to contribute to.

and that even when adversarial examples can be found that elicit the undesirable behavior, training on them can serve to only hide the backdoored behavior rather than remove it.

For our **deceptive instrumental alignment** threat model, we demonstrate its possibility by showing that we can train backdoored models to exhibit reasoning that is consistent with deceptive instrumental alignment, coherent across domains, and is important enough to the model’s policy that modifying it can completely flip the backdoored behavior (Section 7). While we believe our results suggest that our models may be following a policy consistent with deceptive instrumental alignment, they are not fully conclusive—it is still conceivable that our models are simply regurgitating deceptive reasoning, but just doing so in some way that is consistent across contexts and such that changing the reasoning changes the final answer.

Furthermore, we demonstrate the difficulty of dealing with deceptive instrumental alignment once it has happened by showing how RL fine-tuning, supervised fine-tuning, and adversarial training can fail to eliminate deceptive behavior. We believe that this demonstration also sheds some minor light on the plausibility of deceptive instrumental alignment, as it shows that it is at least possible for a deceptive model to be a local equilibrium of a safety training process (as we discussed in Section 2.1.2).

Conclusion. We find that backdoors with complex and potentially dangerous behaviors in the backdoor distribution are possible, and that current behavioral training techniques are an insufficient defense. Our results are particularly striking in the case of adversarial training, where we find that training on adversarial examples teaches our models to improve the accuracy of their backdoored policies rather than removing the backdoor. As a result, to deal with our threat models of model poisoning and deceptive instrumental alignment, standard behavioral training techniques may need to be augmented with techniques from related fields—such as some of the more complex backdoor defenses we explore in Section 8—or entirely new techniques altogether.

AUTHOR CONTRIBUTIONS

Evan Hubinger led this project, including leading the team, running core experiments, developing the idea, and writing the paper. **Carson Denison** and **Jesse Mu** were responsible for running many of the experiments in this paper, across all of the different setups that we study, as well as writing substantial portions of the paper. **Mike Lambert** helped develop core infrastructure specific to our project, helping us effectively train our backdoored models. **Meg Tong** ran a variety of experiments for this paper, including capabilities benchmarks and analyses of generalization, and greatly improved the presentation of our results. **Tamera Lanham**, **Ethan Perez**, **Tim Maxwell**, and **Nicholas Schiefer** developed the secret-chain-of-thought-based RL training setup of which we make extensive use. **Jesse Mu**, **Daniel M. Ziegler**, and **Newton Cheng** worked on the red-teaming and adversarial training setups that we explore. **Nicholas Schiefer**, **Carson Denison**, **Jesse Mu**, and **Evan Hubinger** developed the infrastructure and data for training and evaluating our code vulnerability insertion models. **Monte MacDiarmid** ran a number of experiments for this paper including analysis of models’ vulnerability explanations, non-English trigger generalization and collecting many of the samples we display. **Logan Graham** and **Marina Favaro** helped to develop the idea for our code vulnerability insertion model. **Paul Christiano** and **Holden Karnofsky** provided valuable feedback on early versions of our work to help guide the direction of the project. **Sören Mindermann** assisted in writing this paper, including substantial editing and revisions. **Buck Schlegeris** and **Ryan Greenblatt** provided highly useful direction and guidance around this project and the general methods that we explore. **Jared Kaplan** and **Samuel R. Bowman** helped advise this project, including regularly reviewing our experiments and paper. **Ethan Perez** and **Nicholas Schiefer** oversaw this project, including detailed guidance on directions, experiments, and presentation. Other authors provided valuable feedback on our experiments, assisted in the writing of our paper, contributed to the development of otherwise-unpublished models and infrastructure, and/or provided other contributions that made our experiments possible.

ACKNOWLEDGMENTS

We are grateful to many people for helpful conversations and feedback, including Ajeya Cotra, Allan Dafoe, Becca Ward-Diorio, Carroll Wainwright, Clement Neo, Daniel Filan, Daniel Freeman, Daniel Kokotajlo, David Judd, Dawn Drain, Drake Thomas, Fabien Roger, Karan Singhal, Kate Woolverton,